

Strassen's Matrix Multiplication Algorithm

Sreejit Maity

July 30, 2026

1 Introduction

Matrix multiplication is one of the most fundamental operations in numerical computing, computer graphics, machine learning, and scientific simulation. The naive algorithm for multiplying two $n \times n$ matrices computes each entry of the product as a sum of n products, resulting in an overall time complexity of $O(n^3)$. For a long time this cubic bound was believed to be essentially unavoidable, until in 1969 the German mathematician Volker Strassen showed that matrix multiplication could be performed asymptotically faster than the schoolbook method. His algorithm reduces the complexity to approximately $O(n^{2.807})$, where $2.807 = \log_2 7$.

2 The Idea Behind Strassen's Algorithm

Strassen's key insight was that two 2×2 matrices can be multiplied using only **7** scalar multiplications instead of the 8 required by the straightforward definition of matrix multiplication. Since multiplication is computationally more expensive than addition (especially for large matrices, where a "scalar" is itself a submatrix), reducing the number of multiplications, even at the cost of a few extra additions, yields an asymptotic speedup.

Given two 2×2 matrices

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}, \quad B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix},$$

Strassen defines seven intermediate products:

$$M_1 = (a_{11} + a_{22})(b_{11} + b_{22})$$

$$M_2 = (a_{21} + a_{22})b_{11}$$

$$M_3 = a_{11}(b_{12} - b_{22})$$

$$M_4 = a_{22}(b_{21} - b_{11})$$

$$M_5 = (a_{11} + a_{12})b_{22}$$

$$M_6 = (a_{21} - a_{11})(b_{11} + b_{12})$$

$$M_7 = (a_{12} - a_{22})(b_{21} + b_{22})$$

The entries of the product matrix $C = A \times B$ are then recovered purely through addition and

subtraction of these seven products:

$$\begin{aligned}c_{11} &= M_1 + M_4 - M_5 + M_7 \\c_{12} &= M_3 + M_5 \\c_{21} &= M_2 + M_4 \\c_{22} &= M_1 - M_2 + M_3 + M_6\end{aligned}$$

By recursively applying this decomposition to larger matrices (splitting an $n \times n$ matrix into four $n/2 \times n/2$ blocks), the recurrence for the running time becomes

$$T(n) = 7T(n/2) + O(n^2),$$

which by the Master Theorem solves to $T(n) = O(n^{\log_2 7}) \approx O(n^{2.807})$, an improvement over the $O(n^3)$ of the standard algorithm.

3 Worked Example

Let

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

Computing the seven products:

$$\begin{aligned}M_1 &= (1 + 4)(5 + 8) = 5 \times 13 = 65 \\M_2 &= (3 + 4) \times 5 = 7 \times 5 = 35 \\M_3 &= 1 \times (6 - 8) = 1 \times (-2) = -2 \\M_4 &= 4 \times (7 - 5) = 4 \times 2 = 8 \\M_5 &= (1 + 2) \times 8 = 3 \times 8 = 24 \\M_6 &= (3 - 1)(5 + 6) = 2 \times 11 = 22 \\M_7 &= (2 - 4)(7 + 8) = -2 \times 15 = -30\end{aligned}$$

Combining these:

$$\begin{aligned}c_{11} &= 65 + 8 - 24 + (-30) = 19 \\c_{12} &= -2 + 24 = 22 \\c_{21} &= 35 + 8 = 43 \\c_{22} &= 65 - 35 + (-2) + 22 = 50\end{aligned}$$

So

$$C = A \times B = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix},$$

which agrees exactly with the result obtained from ordinary 2×2 matrix multiplication, confirming the correctness of the method.

4 C Implementation

The following C program implements Strassen's algorithm for 2×2 matrices, following the formulas derived above.

Listing 1: Strassen's algorithm for 2×2 matrix multiplication in C

```
1 #include <stdio.h>
2
3 /* Multiplies two 2x2 matrices A and B using Strassen's algorithm,
4    storing the result in C. */
5 void strassenMultiply(int A[2][2], int B[2][2], int C[2][2]) {
6     int M1, M2, M3, M4, M5, M6, M7;
7
8     M1 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
9     M2 = (A[1][0] + A[1][1]) * B[0][0];
10    M3 = A[0][0] * (B[0][1] - B[1][1]);
11    M4 = A[1][1] * (B[1][0] - B[0][0]);
12    M5 = (A[0][0] + A[0][1]) * B[1][1];
13    M6 = (A[1][0] - A[0][0]) * (B[0][0] + B[0][1]);
14    M7 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);
15
16    C[0][0] = M1 + M4 - M5 + M7;
17    C[0][1] = M3 + M5;
18    C[1][0] = M2 + M4;
19    C[1][1] = M1 - M2 + M3 + M6;
20 }
21
22 int main(void) {
23     int A[2][2] = {{1, 2}, {3, 4}};
24     int B[2][2] = {{5, 6}, {7, 8}};
25     int C[2][2];
26
27     strassenMultiply(A, B, C);
28
29     printf("Result matrix C = A x B:\n");
30     for (int i = 0; i < 2; i++) {
31         for (int j = 0; j < 2; j++) {
32             printf("%d ", C[i][j]);
33         }
34         printf("\n");
35     }
36
37     return 0;
38 }
```

Running this program on the example matrices from Section 3 produces the output

Result matrix C = A x B:

19 22

43 50

which matches the hand-computed result.

5 Discussion and Practical Considerations

Although Strassen’s algorithm improves the asymptotic exponent from 3 to $\log_2 7 \approx 2.807$, it is rarely used in practice for small matrices. The extra additions and subtractions introduce constant-factor overhead, and the recursive structure requires additional memory for intermediate submatrices. For this reason, practical implementations typically fall back to conventional multiplication once submatrix sizes drop below some threshold (a technique often called a “crossover point”). Furthermore, Strassen’s method is known to be less numerically stable than the standard algorithm, since it involves subtractions that can amplify floating-point rounding errors.

Nevertheless, Strassen’s algorithm remains historically significant: it was the first demonstration that the naive $O(n^3)$ bound for matrix multiplication is not optimal, and it opened the door to a long line of research culminating in algorithms with even lower asymptotic exponents, such as the Coppersmith–Winograd algorithm and its successors.

6 Conclusion

Strassen’s algorithm is a classic example of how a clever reformulation of a problem, trading multiplications for additions, can yield an asymptotic improvement over a seemingly optimal brute-force method. While practical performance gains only materialize for sufficiently large matrices, the algorithm’s theoretical importance in the study of computational complexity is difficult to overstate.